

\$match (aggregation)

Definition

\$match

Filters the documents to pass only the documents that match the specified condition(s) to the next pipeline stage.

Compatibility

You can use `$match` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

The `$match` stage has the following prototype form:

```
{ $match: { <query> } }
```



`$match` takes a document that specifies the query conditions. The query syntax is identical to the read operation query syntax; i.e. `$match` does not accept raw aggregation expressions. Instead, use a `$expr` query expression to include aggregation expression in `$match`.

Behavior

Pipeline Optimization

- Place the `$match` as early in the aggregation pipeline as possible. Because `$match` limits the total number of documents in the aggregation pipeline, earlier `$match` operations minimize the amount of processing down the pipe.
- If you place a `$match` at the very beginning of a pipeline, the query can take advantage of indexes like any other `db.collection.find()` or `db.collection.findOne()`.

Restrictions

- The `$match` query syntax is identical to the read operation query syntax; i.e. `$match` does not accept raw aggregation expressions. To include aggregation expression in `$match`, use a `$expr` query operator:

```
{ $match: { $expr: { <aggregation expression> } } }
```



- You cannot use `$where` in `$match` queries as part of the aggregation pipeline.
- You cannot use `$near` or `$nearSphere` in `$match` queries as part of the aggregation pipeline. As an alternative, you can either:
 - Use `$geoNear` stage instead of the `$match` stage.
 - Use `$geoWithin` query operator with `$center` or `$centerSphere` in the `$match` stage.
- To use `$text` in the `$match` stage, the `$match` stage has to be the first stage of the pipeline.

Views do not support `$text`.

NOTE

`$text` provides text query capabilities for self-managed (non-Atlas) deployments. For data hosted on MongoDB Atlas, MongoDB offers an improved full-text query solution, Atlas Search.

Filter Data on Atlas by Using Atlas Search

For data stored in MongoDB Atlas, you can use the Atlas Search compound operator `filter` option to match or filter documents when running `$search` queries. Running `$match` after `$search` is less performant than running `$search` with the compound operator `filter` option.

To learn more about the `filter` option, see compound in the Atlas documentation.

Examples

The examples use a collection named `articles` with the following documents:

```
{ "_id" : ObjectId("512bc95fe835e68f199c8686"), "author" : "dave",  
  "score" : 80, "views" : 100 }  
{ "_id" : ObjectId("512bc962e835e68f199c8687"), "author" : "dave",  
  "score" : 85, "views" : 521 }  
{ "_id" : ObjectId("55f5a192d4bede9ac365b257"), "author" : "ahn",  
  "score" : 60, "views" : 1000 }  
{ "_id" : ObjectId("55f5a192d4bede9ac365b258"), "author" : "li", "score"  
  : 55, "views" : 5000 }  
{ "_id" : ObjectId("55f5a1d3d4bede9ac365b259"), "author" : "annT",  
  "score" : 60, "views" : 50 }  
{ "_id" : ObjectId("55f5a1d3d4bede9ac365b25a"), "author" : "li", "score"  
  : 94, "views" : 999 }  
{ "_id" : ObjectId("55f5a1d3d4bede9ac365b25b"), "author" : "ty", "score"  
  : 95, "views" : 1000 }
```



Equality Match

The following operation uses `$match` to perform a simple equality match:

```
db.articles.aggregate(  
  [ { $match : { author : "dave" } } ]  
)
```



The `$match` selects the documents where the `author` field equals `dave`, and the aggregation returns the following:

```
{ "_id" : ObjectId("512bc95fe835e68f199c8686"), "author" : "dave",  
  "score" : 80, "views" : 100 }  
{ "_id" : ObjectId("512bc962e835e68f199c8687"), "author" : "dave",  
  "score" : 85, "views" : 521 }
```

Perform a Count

The following example selects documents to process using the `$match` pipeline operator and then pipes the results to the `$group` pipeline operator to compute a count of the documents:

```
db.articles.aggregate( [  
  { $match: { $or: [ { score: { $gt: 70, $lt: 90 } }, { views: { $gte:  
    1000 } } ] } },  
  { $group: { _id: null, count: { $sum: 1 } } }  
] );
```

In the aggregation pipeline, `$match` selects the documents where either the `score` is greater than `70` and less than `90` or the `views` is greater than or equal to `1000`. These documents are then piped to the `$group` to perform a count. The aggregation returns the following:

```
{ "_id" : null, "count" : 5 }
```

Additional Information

Refer to the following pages for more information and use cases on aggregation.

- [Aggregation with the Zip Code Data Set](#)
- [Aggregation with User Preference Data](#)